



Bilgisayar Programlama

Yapılar (Structures) ve Birliktelikler (Unions)

İçindekiler

1. Giriş ve Motivasyon: Neden "Struct"?
2. Yapı (struct) Tanımlama: Sözdizimi ve Temel Kullanım
3. Bellek Yönetimi: Hizalama (Alignment) ve Padding
4. Karmaşık Yapılar: Diziler ve İç İççe (Nested) Yapılar
5. İşaretçiler ve Fonksiyonlar: Verimli Parametre Aktarımı
6. Birliktelikler (Unions): Bellek Tasarrufu ve Veri Ayrıştırma
7. Bit Sahaları (Bitfields): Donanım Register Erişimi
8. Laboratuvar Uygulamaları



BÖLÜM 1

Giriş

"Struct" (Yapı) Nedir? Neden İhtiyacımız Var?

Şu ana kadar öğrendiğimiz değişken tipleri (`int` , `float` , `char`) tek bir veriyi temsil eder. Diziler ise aynı tipten çok sayıda veriyi gruplar.

Gerçek hayatta bir nesne sadece tek bir tipten oluşmaz.

Örneğin, bir Robot nesnesini düşünelim:

- ID: `int`
- İsim: `char[]`
- Konum (X, Y): `float`
- Pil Durumu: `int`
- Aktif Mi?: `bool`

Bu verileri ayrı ayrı değişkenlerde tutmak (örn: `robot1_id` , `robot1_x` ...) karmaşıklığı artırır ve yönetimi zorlaştırır.

Statik Diziler vs. Yapılar

Diziler "Homojen" (aynı tip), Yapılar ise "Heterojen" (farklı tip) veri koleksiyonlarıdır.

Özellik	Diziler (Arrays)	Yapılar (Structures)
Veri Tipi	Tek tip (int, float vb.)	Karışık tipler (int + float + char...)
Erişim	İndeks ile (<code>dizi[0]</code>)	Üye ismi ile (<code>robot.id</code>)
Analoji	Bir apartmandaki birbirinin aynı daireler	Bir sırt çantasındaki farklı eşyalar (kitap, kalem, su)

Mekatronik Senaryosu: IMU Sensör Paketi

Bir Atalet Ölçüm Birimi (IMU) sensöründen gelen verileri düşünün. Bu veriler ayrılmaz bir bütündür:

- İvmeölçer: x, y, z (`float`)
- Jiroskop: roll, pitch, yaw (`float`)
- Zaman Damgası: `unsigned long`

Bu verileri tek bir "paket" (`struct`) içinde tutmak, verinin bütünlüğünü sağlar ve fonksiyonlara tek bir "nesne" olarak gönderilmesine imkan tanır.



BÖLÜM 2

Yapı (struct) Tanımlama ve Kullanma

Temel Sözdizimi (Syntax)

Bir yapı tanımlamak için `struct` anahtar kelimesi kullanılır. Bu işlem aslında yeni bir veri tipi tanımlamaktır.

```
1 // Yapı tanımı (Genellikle main dışında yapılır)
2 struct Robot {
3     int id;           // Üye (Member) 1
4     float x_konum;    // Üye 2
5     float y_konum;    // Üye 3
6     int pil;          // Üye 4
7 }; // Noktalı virgüle DİKKAT!
8
9 int main() {
10     // Yapıdan değişken (nesne) oluşturma
11     struct Robot robot1;
12
13     return 0;
14 }
```


Üye Erişimi: Nokta (.) Operatörü

Yapı içerisindeki elemanlara erişmek için nokta operatörü kullanılır.

```
1  #include <stdio.h>
2
3  struct Robot {
4      int id;
5      int pil;
6  };
7
8  int main() {
9      struct Robot r1;
10
11     r1.id = 101;        // Atama
12     r1.pil = 85;
13
14     printf("Robot ID: %d, Pil: %%d\n", r1.id, r1.pil);
15
16     return 0;
17 }
```

Yapı Başlatma (Initialization)

Yapılara ilk değerlerini tanımlama anında iki farklı yöntemle verebiliriz:

```
1 // 1. Sıralı Başlatma (Liste Yöntemi)
2 struct Robot r1 = {101, 15.5, 20.0, 100};
3
4 // 2. Belirlenmiş Atama (Designated Initializer - C99 ve sonrası) ✓
5 // Mekatronik uygulamalarında daha güvenli ve okunaklıdır!
6 struct Robot r2 = {
7     .id = 102,
8     .pil = 95,
9     .x_konum = 0.0,
10    .y_konum = 5.0
11 };
```

💡 Nokta (.) Niçin Var ve Avantajları Nelerdir?

- Doğrudan Eşleme: Nokta işareti, struct içerisindeki üyelere isimleriyle doğrudan atıfta bulunmayı sağlar (`.id` gibi).
- Sıra Bağımsızlığı: Sıralı başlatmanın aksine üyelerin sırası değişse bile kod kırılmaz veya yanlış atama yapılmaz.
- Mekatronik Güvenliği: Sensör/motor yapılandırması gibi çok sayıda parametre içeren büyük struct'larda hata yapmayı engeller. Değer atanmayan üyeler otomatik olarak `0` veya `0.0` değerini alır.

```
1  USART_Config_t config = {  
2      .baud_rate = 9600,  
3      .data_bits = 8,  
4      .stop_bits = 1,  
5      .parity = PARITY_NONE  
6  };
```

typedef Kullanımı

Her seferinde `struct Robot` yazmak yerine, `typedef` ile daha kısa ve şık isimler kullanabiliriz.

```
1  typedef struct {
2      int id;
3      float sicaklik;
4  } Sensor_t; // _t soneki (suffix) 'type' anlamına gelir
5
6  int main() {
7      // Artık 'struct' kelimesine gerek yok
8      Sensor_t s1 = {1, 24.5};
9
10     return 0;
11 }
```

Yapıların Birbirine Atanması (Shallow Copy)

Aynı tipten iki yapı değişkeni birbirine doğrudan atanabilir. Bu işlem tüm üyeleri bellekteki sırasıyla kopyalar.

```
1  Sensor_t s1 = {5, 30.2};  
2  Sensor_t s2;  
3  
4  s2 = s1; // s1'in tüm üyeleri s2'ye kopyalandı!  
5  
6  printf("S2 ID: %d, Sicaklik: %.1f\n", s2.id, s2.sicaklik);
```

Örnek Kod: Motor Parametreleri

```
1  #include <stdio.h>
2
3  typedef struct {
4      int motor_id;
5      int rpm;
6      float voltaj;
7      float akim;
8  } Motor_t;
9
10 int main() {
11     Motor_t m1 = {.motor_id = 1, .rpm = 3000, .voltaj = 12.0, .akim = 1.5};
12
13     printf("Motor %d Durumu:\n", m1.motor_id);
14     printf("Hiz: %d RPM, Guc: %.2f Watt\n", m1.rpm, m1.voltaj * m1.akim);
15
16     return 0;
17 }
```



BÖLÜM 3

Bellek Yönetimi, Hizalama ve Padding

Problem: sizeof Neden 8?

```
1 typedef struct {  
2     char a;    // 1 byte  
3     int b;     // 4 byte  
4 } Test_t;
```

Toplam boyut: $1 + 4 = 5$ byte beklenir, ancak `sizeof(Test_t) = 8` byte çıkar. Neden?

Cevap: Bellek Hizalama (Alignment) ve Padding (Dolgu)

Kavram: Bellek Hizalama (Alignment)

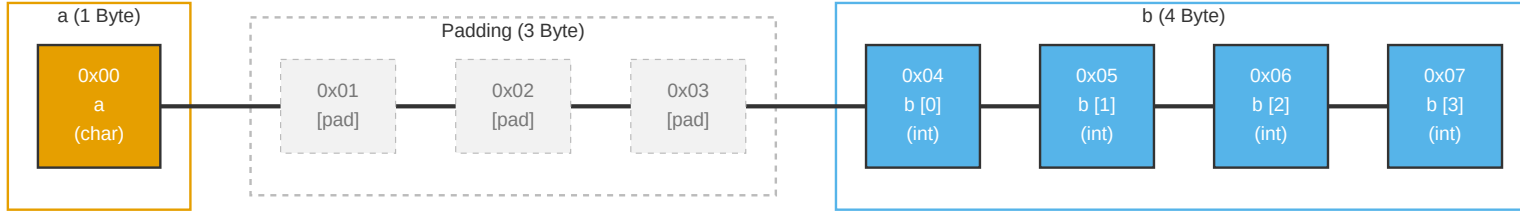
İşlemci belleği kelimeler (word) halinde okur — tipik olarak 4 byte'lık bloklar. Bir `int` (4 B) bu bloklardan birine tam sığar.

Veri Tipi	Boyut	Başlangıç Adresi (Kural)
<code>char</code>	1 B	Herhangi bir adres
<code>short</code>	2 B	Çift sayı (2'nin katı)
<code>int</code> , <code>float</code>	4 B	4'ün katı (0, 4, 8, 12...)
<code>double</code>	8 B	8'in katı (0, 8, 16...)

⚠ `int` değişkenini adres 0x01'e koyarsanız, 4 byte'lık veri iki farklı kelimeye yayılır — işlemci iki ayrı okuma yapmak zorunda kalır.

Bellek Yerleşimi:

Aşağıdaki diyagram `Test_t` struct'ının bellekteki 8 byte'lık yatay dizilimini gösterir:



💡 `char a (1 B) → 0x00'e` yerleşir. `int b (4 B)` `0x01'den` başlayamaz (4'ün katı değil). Derleyici performansı korumak için `0x01-0x03` arasına padding ekler ve `b` 'yi `0x04'e` hizalar.

Padding: Derleyicinin Çözümü

Derleyici, her üyeyi kendi hizalama kuralına uygun adrese yerleştirir. Gerekirse araya dolgu (padding) ekler.

```
1 typedef struct {  
2     char a;      // 1 B → adres 0x00 (✓)  
3     // [3 B padding] → 0x01-0x03 (boş)  
4     int b;       // 4 B → adres 0x04 (✓ 4'ün katı)  
5 } Test_t;       // sizeof = 8 B
```

Üye	Boyut	Başlangıç Adresi	Kullanılan Byte'lar
a (char)	1	0x00	0x00
Padding	3	0x01	(boş)
b (int)	4	0x04	0x04-0x07
Toplam	5		8 B (3 B padding)

Neden Padding? — Performans Sebebi

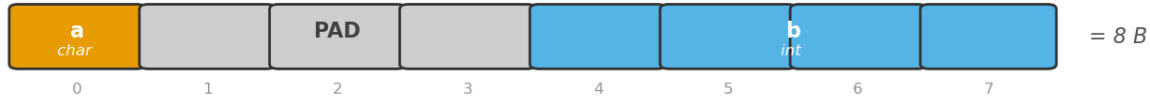
Hizalama olmazsa CPU aynı veriyi okumak için 2 bellek erişimi yapmak zorunda kalır:

✓ Hızlı (`int` 0x04'te): `[b][b][b][b]` → tek okuma × Hızsız (`int` 0x01'de): `[b][b]` + `[b][b]` → iki okuma

⚠ Gömülü sistemlerde (ARM Cortex-M, AVR) hizasız erişim **HardFault** ile programın çökmesine yol açar!

Padding vs Packed: Karşılaştırma

struct Test_t { char a; int b; } → sizeof = 8 B (3 B padding)



struct Paket_t { char id; int veri; } → #pragma pack(1) ile sizeof = 5 B



Legend: ■ char (1 B) ■ int (4 B) ■ Padding (bos)

Dikkat: Üye Sıralaması Önemlidir!

Aynı üyeleri farklı sırayla tanımlamak, struct'ın toplam boyutunu değiştirir!

```
1 // ❌ KÖTÜ: Büyükten küçüğe sıralanmamış
2 typedef struct {
3     char c;        // 1 B → adres 0x00
4     // [3 B padding]      0x01-0x03 ← BOŞA GİDEN ALAN!
5     int i;         // 4 B → adres 0x04
6     short s;       // 2 B → adres 0x08
7     // [2 B padding]      0x0A-0x0B ← BOŞA GİDEN ALAN!
8 } Kotu_t;          // sizeof = 12 B (sadece 7 B kullanılır, 5 B kayıp)
```

Üye Sıralaması:

```
1 // ✅ İYİ: Büyükten küçüğe sıralanmış (padding minimize)
2 typedef struct {
3     int i;        // 4 B → adres 0x00
4     short s;      // 2 B → adres 0x04
5     char c;       // 1 B → adres 0x06
6     // [1 B padding]      0x07      ← sadece 1 B kayıp!
7 } İyi_t;          // sizeof = 8 B (7 B kullanılır, 1 B kayıp)
```

Yöntem	Sıralama	sizeof	Kayıp
× Kötü	char → int → short	12 B	5 B
✅ İyi	int → short → char	8 B	1 B

⚠️ Aynı veriler, sadece sıralama farkıyla %33 daha az bellek!

Üye Sıralama Kuralı

Temel Kural: Üyeleri büyükten küçüğe (azalan boyut) sıralayın.

Öncelik	Veri Tipleri	Boyut
1 (en büyük)	<code>double</code> , <code>long long</code>	8 B
2	<code>int</code> , <code>float</code> , <code>long</code>	4 B
3	<code>short</code>	2 B
4 (en küçük)	<code>char</code> , <code>bool</code>	1 B

Örnek: Çoklu veri tipi içeren struct

```
1
2  typedef struct {
3      // 1. En büyükler
4      double sicaklik; // 8 B
5      // 2. Orta boy
6      int     basinc;   // 4 B
7      int     nem;      // 4 B
8      // 3. Küçükler
9      short   ruzgar_hiz; // 2 B
10     char     sensor_id; // 1 B
11     // [1 B padding]
12 } Meteoroloji_t; // sizeof = 24 B (23 kullanılır, 1 B kayıp)
```

💡 Elle sıralama kuralı: double → int / float → short → char

Mekatronik Notu: Haberleşme Paketleri

Eğer bu struct'ı UART veya SPI üzerinden başka bir cihaza (örn: Arduino'dan PC'ye) gönderiyorsanız, padding veriyi bozar! Karşı taraf padding'i bilmezse yanlış baytları okur.

✓ Çözüm: `#pragma pack(1)` Derleyiciye "sıkıştır" (pack) emri vererek padding'i kaldırabiliriz.

```
1  #pragma pack(push, 1) // Sıkıştırmayı başlat (1 byte hizalama)
2  typedef struct {
3      char id;
4      int veri;
5  } Paket_t;
6  #pragma pack(pop) // Eski ayara dön
```

`sizeof(Paket_t)` artık tam olarak 5 byte çıkacaktır.

Not: Byte Sırası — Little-Endian vs Big-Endian

Padding'i hallettiniz, peki ya byte sırası? Aynı 4 byte farklı işlemcilerde ters okunabilir!

```
1  typedef union {  
2      float deger;  
3      unsigned char bayt[4];  
4  } Sensor_t;  
5  
6  Sensor_t s;  
7  s.deger = 25.0; // 0x41C80000 IEEE 754  
8  
9  printf("%02X %02X %02X %02X\n", s.bayt[0], s.bayt[1], s.bayt[2], s.bayt[3]);
```

Byte Sırası

Sistem	Mimariler	Byte Sırası (25.0 = 0x41C80000)
Little-Endian	ARM, x86, AVR (Arduino)	00 00 C8 41 (ters)
Big-Endian	SPI sensörler, TCP/IP, Motorola	41 C8 00 00 (düz)

UART/SPI ile haberleşirken, aynı `float` değeri karşı tarafta anlamsız bir sayı olarak okunabilir!

✓ Çözüm: Gönderirken/alırken byte sırasını elle düzenleyin veya `htonl()` / `ntohl()` fonksiyonlarını kullanın.

BÖLÜM 4

Karmaşık Yapılar: Dizi ve İç İç Yapılar

Yapı Dizileri (Arrays of Structs)

Aynı tipten çok sayıda nesneyi (örn: bir sürü sensör) yönetmek için dizi kullanırız.

```
1  Sensor_t sensorler[10]; // 10 adet sensör nesnesi
2
3  sensorler[0].id = 1;
4  sensorler[0].sicaklik = 25.5;
5
6  sensorler[1].id = 2;
7  sensorler[1].sicaklik = 26.1;
```

İç İçe Yapılar (Nested Structs)

Bir yapının üyesi, başka bir yapı olabilir. Bu, karmaşık sistemleri hiyerarşik olarak modellememizi sağlar.

```
1  typedef struct {
2      float x;
3      float y;
4  } Koordinat_t;
5
6  typedef struct {
7      int id;
8      Koordinat_t konum; // İç içe yapı
9      int pil;
10 } Robot_t;
11
12 // Kullanım:
13 Robot_t r1;
14 r1.konum.x = 10.5;
15 r1.konum.y = 20.0;
```

Veri Hiyerarşisi Örneği

Mekatronik Uygulaması: Bir dronun tüm alt sistemlerini tek bir ana yapıda toplayabiliriz.

```
1  typedef struct {  
2      Koordinat_t hedef;  
3      Koordinat_t guncel;  
4      float yukseklik;  
5  } Navigasyon_t;  
6  
7  typedef struct {  
8      Navigasyon_t nav;  
9      Motor_t motorlar[4];  
10     int batarya;  
11 } Drone_t;
```




BÖLÜM 5

Yapılar, İşaretçiler ve Fonksiyonlar

Yapı İşaretçileri (Struct Pointers)

Bir yapının adresini tutan işaretçiler, mekatronikte performans için zorunluluktur.

```
1 Robot_t r1 = {.id = 1, .pil = 100};
2 Robot_t *ptr = &r1;
3
4 // İşaretçi üzerinden erişim (Zor yol)
5 (*ptr).id = 2;
6
7 // İşaretçi üzerinden erişim (OK OPERATÖRÜ - ->) ✓
8 ptr->id = 2;
```

⚠ Ok Operatörü (`->`): İşaretçinin gösterdiği yerdeki üyeye erişmek için kullanılır.

Fonksiyonlara Parametre Aktarımı

1. Değer ile (Pass by Value): Yapı kopyalanır. Yavaştır ve çok bellek harcar. 2. Adres ile (Pass by Reference): Sadece adres (8 byte) gönderilir. Hızlıdır ve asıl veriyi değiştirebiliriz.

```
1 // Referans ile çağrı (ADRES GÖNDERİLİR) ✓
2 void Motor_Durdur(Motor_t *m) {
3     m->rpm = 0; // Asıl motoru durdurur
4 }
5
6 int main() {
7     Motor_t motor1 = {.motor_id = 1, .rpm = 2500};
8     Motor_Durdur(&motor1); // Adres gönderdik
9     return 0;
10 }
```

Yapıların Dinamik Tahsisi (malloc)

```
1 Robot_t *r_ptr;  
2  
3 // Bellekte bir Robot_t boyutu kadar yer ayır  
4 r_ptr = (Robot_t *) malloc(sizeof(Robot_t));  
5  
6 if (r_ptr != NULL) {  
7     r_ptr->id = 505;  
8     free(r_ptr); // İş bitince temizle!  
9 }
```

Geri Dönüş Değeri: Struct Döndürme

Fonksiyonlar, geri dönüş tipi olarak `int` veya `float` gibi struct (değer ile) veya struct pointer (adres ile) döndürebilir.

? Temel Soru: C dilinde bir fonksiyon `return` ile aynı anda birden fazla veri döndürebilir mi?

💡 Cevap: Normal şartlarda hayır (fonksiyonlar sadece tek bir değişken dönebilir). Ancak struct kullanarak, farklı türden onlarca veriyi tek bir "paket" haline getirip tek seferde geri döndürebiliriz!

🔧 Mekatronikte Önemi: Bir sensör okuma fonksiyonunun hem sıcaklığı (`float`), hem nemi (`float`), hem de hata kodunu (`int`) aynı anda geri döndürmesini istiyorsanız, en profesyonel yol bu verileri tek bir struct içinde paketleyip döndürmektir.

1. Değer ile Döndürme (Struct by Value) ⚠

```
1  #include <stdio.h>
2  typedef struct {
3      int x, y;
4  } Koordinat_t;
5
6  // ORJINALI KOPYALAR – yavaş, güvenli ✓
7  Koordinat_t Koordinat_Uret(int a, int b) {
8      Koordinat_t yeni = {a, b};
9      return yeni; // Tüm struct kopyalanır
10 }
11
12 int main() {
13     Koordinat_t nokta = Koordinat_Uret(5, 10);
14     printf("(%d, %d)\n", nokta.x, nokta.y); // (5, 10)
15     return 0;
16 }
```

 Ne olur? `return yeni;` çalıştığında struct'ın tüm üyeleri çağırın fonksiyona kopyalanır. Büyük struct'lar için yavaştır.

2. Pointer ile Döndürme ✓ (ÖNERİLEN)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      float sicaklik;
6      float nem;
7  } SensorVeri_t;
8
9  // Pointer döndürür - hızlı ✓
10 SensorVeri_t* Sensor_Oku(float sicaklik, float nem) {
11     SensorVeri_t *s = malloc(sizeof(SensorVeri_t));
12     if (s == NULL) return NULL;
13     s->sicaklik = sicaklik;
14     s->nem = nem;
15     return s; // Sadece adres (8 B) kopyalanır
16 }
17
18 int main() {
19     SensorVeri_t *v = Sensor_Oku(25.5, 60.0);
20     if (v != NULL) {
```

✓ Sadece adres (8 B) kopyalanır — büyük struct'larda çok daha hızlıdır.

5.1 Kritik Uyarı: Yerel Değişkene Pointer Döndürme! x

```
1 // ❌ TEHLİKELİ! Yerel değişkenin adresini döndürme!  
2 int* Hata() {  
3     int yerel = 42;  
4     return &yerel; // Fonksiyon bitince 'yerel' silinir!  
5 } // ← Bu noktada yerel değişken yok oldu!  
6  
7 // ✅ Aynı kural struct pointer için de geçerlidir:  
8 Koordinat_t* Koordinat_Uret_Hatali(int x, int y) {  
9     Koordinat_t gecici = {x, y};  
10    return &gecici; // ❌ gecici fonksiyon sonunda silinir!  
11 }
```

Yöntem	Kopyalanan	Hız	Güvenlik	Kullanım Alanı
Değer (Value)	Tüm struct	x Yavaş	✅ Güvenli	Küçük struct'lar (< 32 B)
Pointer (Heap)	8 B (adres)	✅ Hızlı	⚠️ free() gerekli	Büyük struct'lar, diziler
Pointer (Stack)	8 B (adres)	x Geçersiz!	x Tehlikeli!	ASLA KULLANMA!

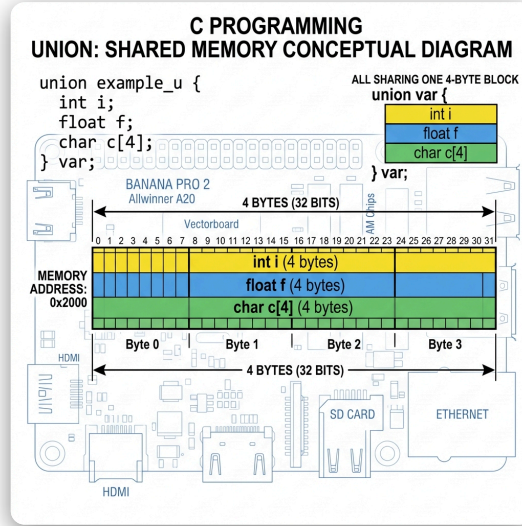


BÖLÜM 6

Birliktelikler (Unions) ve Bit Sahaları

Union Nedir?

`union` , `struct` 'a benzer ancak kritik bir farkı vardır: Tüm üyeler aynı bellek alanını paylaşır.



Union: Bellek Paylaşımı

```
1 union Veri {  
2     int i;  
3     float f;  
4     char c;  
5 };
```

`sizeof(union Veri)` sonucu, en büyük üye olan 4 byte çıkacaktır. Bir üyeye değer atandığında, diğer üyelerin üzerine yazılır.

Union: Bellek Paylaşımı

Union'ın çalışma mantığını anlamak için en temel örnek: Bir tam sayı (`int`) ve bir karakterin (`char`) aynı bellek adresini paylaşmasıdır.

```
1  typedef union {
2      int sayi;
3      char karakter;
4  } Kolay_t;
5
6  int main() {
7      Kolay_t veri;
8
9      veri.sayi = 65; // Tam sayı ataması yapıyoruz
10
11     // Bellekte aynı kutuyu paylaştıkları için:
12     printf("Sayi: %d\n", veri.sayi);           // Çıktı: 65
13     printf("Karakter: %c\n", veri.karakter);    // Çıktı: A (ASCII 65 = 'A')
14
15     return 0;
16 }
```

Arka Planda Ne Oluyor?

Bellek, ortak bir havuz gibidir. `union` içindeki her iki üye de tam olarak aynı başlangıç adresine yerleşir.

Bellek Hücresi Görünümü (ASCII Şeması) 🧠

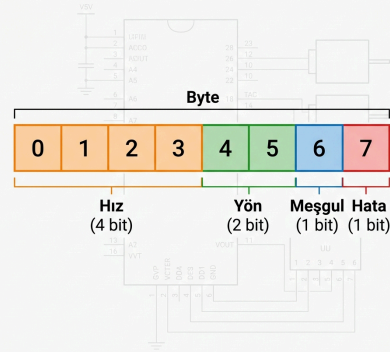
1	Adres:	0x00	0x01	0x02	0x03	
2		+-----+	+-----+	+-----+	+-----+	
3	veri.sayi	65	0	0	0	-> 4 Byte (int)
4		+-----+	+-----+	+-----+	+-----+	
5	veri.karakter	'A'				-> 1 Byte (char)
6		+-----+				

1. `veri.sayi = 65` atandığında, `int` tipi 4 byte kapladığı için `0x00` - `0x03` adres aralığına yerleşir. Little-endian (düşük anlamlı baytın başa yazılması) kuralına göre `65` değeri tam olarak `0x00` adresindeki hücreye yazılır.
2. `veri.karakter` ise sadece 1 byte kaplar ve doğrudan başlangıç adresi olan `0x00` hücresini paylaşır.
3. Sonuç olarak, `veri.karakter` hücresini okuduğumuzda, matematiksel hiçbir işlem yapmadan doğrudan `0x00` adresindeki `65` sayısını okumuş oluruz. Bu sayının `%c` formatındaki ASCII karşılığı ise `'A'` harfidir!

Bit Sahaları (Bitfields)

Bit sahaları, gömülü sistemlerde ve donanım arayüzlerinde bellek tasarrufu sağlamak ve register'lara daha kolay erişmek için kullanılır.

BİT SAHALARI (BITFIELDS) - BELLEK TASARRUFU



Mekatronik Uygulamaları

1. Status Register'ları

- Donanım cihazlarının durumunu kontrol etmek için kullanılır.
- Örnek: Bir motor sürücüsünün durumunu takip etmek için kullanılabilir.

2. Hata Register'ları

- Hata kodlarını bit düzeyinde saklamak için kullanılır.
- Örnek: Bir sensörün farklı hata durumlarını ayrı ayrı bitlerde saklayabilir.

3. Konfigürasyon Register'ları

- Donanım cihazlarının davranışını kontrol etmek için kullanılır.
- Örnek: Bir iletişim protokolünün parametrelerini bit düzeyinde yapılandırmak için kullanılabilir.

Bit Sahaları (Bitfields)

Bir değişkene kaç bit ayrılacağını belirleyebiliriz. Donanım register'larına erişirken bellekten muazzam tasarruf sağlar.

```
1 struct StatusRegister {
2     unsigned int error_flag : 1; // 1 bit
3     unsigned int busy_flag : 1; // 1 bit
4     unsigned int mode      : 3; // 3 bit (0-7 arası değer saklayabilir)
5     unsigned int reserved  : 3; // 3 bit boşluk (toplam 8 bit = 1 byte)
6 };
7
8 int main() {
9     struct StatusRegister motor;
10
11     motor.error_flag = 0; // Hata yok (0)
12     motor.busy_flag = 1; // Sistem meşgul/aktif (1)
13     motor.mode = 5;      // 5 modunu seç (3 bit olduğu için 0-7 arası geçerli)
14
15     printf("Rovut: %lu byte\n", sizeof(motor)); // Sadece 1 byte ver kanları
```


Örnek Bit Decimal Tablosu

Bit decimal tablosu, bit düzeyindeki verilerin ondalık değerlerini göstermek için kullanılır.

Bit	Değer	Bit	Değer	Bit	Değer	Bit	Değer
0000	0	0100	4	1000	8	1100	12
0001	1	0101	5	1001	9	1101	13
0010	2	0110	6	1010	10	1110	14
0011	3	0111	7	1011	11	1111	15



BÖLÜM 7

Laboratuvar Çalışmaları

Uygulama 1: Robot Envanteri

Soru: 5 robotluk bir envanter oluşturun. Her robotun ID ve Pil bilgisini tutun. Pil seviyesi %20'nin altında olan robotları ekrana yazdıran fonksiyonu yazın.

```
1  #include <stdio.h>
2
3  typedef struct {
4      int id;
5      int pil;
6  } Robot_t;
7
8  void Kontrol_Et(Robot_t *liste, int n) {
9      for(int i=0; i<n; i++) {
10         if(liste[i].pil < 20) {
11             printf("⚠ Robot %d kritik pil! (%%d)\n", liste[i].id, liste[i].pil);
12         }
13     }
14 }
```

```
1  ⚠ Robot 102 kritik pil! (%15)
2  ⚠ Robot 104 kritik pil! (%5)
```

Uygulama 2: IMU Veri Ayırıştırma (Union)

Soru: UART'tan gelen 4 baytlık ham veriyi union kullanarak `float` sıcaklık değerine dönüştüren programı yazın. Gelen baytlar: `0x41, 0x20, 0x00, 0x00` (25.0 değerine karşılık gelir).

```
1  #include <stdio.h>
2
3  typedef union {
4      float sicaklik;
5      unsigned char baytlar[4];
6  } SensorPaketi_t;
7
8  int main() {
9      SensorPaketi_t p;
10
11      // UART'tan gelen baytlar (örnek: 25.0 C)
12      p.baytlar[0] = 0x00;
13      p.baytlar[1] = 0x00;
14      p.baytlar[2] = 0x20;
```

```
1  Gelen Float Deger: 25.00 C
```

- Union'a yeni bir üye olarak `int` ekleyip aynı bellek alanını nasıl paylaştığını gözlemleyin

Uygulama 3: Motor Test İstasyonu

Soru: 3 adet DC motorun (her biri için: ID, RPM, akım, sıcaklık) parametrelerini struct dizi ile tutan ve tüm motorları sırayla test eden programı yazın. Motor sıcaklığı 80°C üzerinde ise uyarı versin.

```
1  #include <stdio.h>
2
3  typedef struct {
4      int id;
5      int rpm;
6      float akim;
7      float sicaklik;
8  } Motor_t;
9
10 void TestEt(Motor_t *m) {
11     Motor 1: 3000 RPM, 1.5 A, 45.2 C
12     Motor 2: 2500 RPM, 2.1 A, 82.7 C
13     ⚠ ASIRI ISINMA! Sogutma baslatiliyor...
14     Motor 3: 2800 RPM, 1.8 A, 63.0 C
```

- Motor sayısını kullanıcıdan alarak dinamik dizi (`malloc`) ile oluşturun
- Ortalama akım ve sıcaklık değerlerini hesaplayan fonksiyon ekleyin

Yapılar (Structures) ve Birliktelikler (Unions)

Uygulama 4: Drone Navigasyon Sistemi (İç İçe Struct)

Soru: Bir dronun navigasyon bilgilerini (hedef koordinat, güncel koordinat, yükseklik) ve 4 motorunu iç içe struct kullanarak modelleyin. Tüm sistemi başlatan ve ekrana yazdıran fonksiyonu yazın.

```
1  #include <stdio.h>
2
3  typedef struct {
4      float x;
5      float y;
6  } Koordinat_t;
7
8  typedef struct {
9      int rpm;
10 } Motor_t;
11
12 typedef struct {
```

Beklenen Çıktı:

```
1 Drone Durumu:  
2   Hedef: (100.0, 200.0)  
3   Guncel: (10.0, 15.0)  
4   Yukseklik: 50.0 m  
5   Motor 1: 3000 RPM  
6   Motor 2: 3000 RPM  
7   Motor 3: 3000 RPM  
8   Motor 4: 3000 RPM
```

- Her döngüde drone koordinatını hedefe 1 birim yaklaştıran `Guncelle()` fonksiyonu ekleyin
- Pil seviyesi (`int batarya`) üyesini de struct'a ekleyip iniş mantığı oluşturun

Özet

Bu Hafta İşlenen Konular:

- Yapılar (struct): Heterojen verileri tek bir çatı altında toplama
- Bellek Hizalama ve Padding: Struct'ların gerçek boyutunu anlama
- İç İç Yapılar: Karmaşık sistemleri hiyerarşik modelleme
- Yapı İşaretçileri: `->` operatörü ve verimli parametre aktarımı
- Birliktelikler (union): Bellek paylaşımı ve veri ayrıştırma
- Bit Sahaları: Donanım register'larına bit düzeyinde erişim

İyi Pratikler:

1. ✓ `typedef` kullanın: Kodunuz daha temiz ve modern görünür.
2. ✓ Pointer ile gönderin: Yapıları fonksiyonlara gönderirken `*` kullanın.
3. ✓ `free()` unutmayın: Dinamik ayrılan her yapıyı serbest bırakın.
4. ⚠ Padding dikkat: Haberleşme yapıyorsanız `#pragma pack(1)` kullanın.
5. ⚠ Union kullanımı: Sadece aynı anda tek veri aktifse kullanın.

Teşekkürler...

Dr. Öğr. Üyesi Hüseyin YÜCE

Marmara Üniversitesi

Mekatronik Mühendisliği Bölümü